# ERP Data Migration Tool

User Guide

Copy data from the old ERP export into the new ERP's column layout — safely, and without typing anything by hand.

A step-by-step guide — no programming experience needed

# 1 What This Tool Does

This tool copies data from the **old ERP's** Excel export into a brand-new Excel file laid out the way the **new ERP** expects it. You tell it, once, which old column goes into which new column — the tool does the copying.

## How it works, in plain terms

1. You open a small Excel file called **data_migration_helper.xlsx** and write down which old column name matches which new column name.
2. You run the tool and point it at your old ERP export file.
3. The tool reads the mapping, creates a brand-new Excel file, and copies every value across exactly as it is — dates stay dates, codes keep their leading zeros, and numbers are never rounded.
4. Before it saves anything, it double-checks every single copied cell against the old file. If even one value doesn't match, it stops and creates nothing.

> **The old file is never touched**
>
> The tool only *reads* the old ERP file and the mapping file. It never changes them, and it never overwrites an existing new file unless you explicitly say so.

## What's in this folder

- `main.py` — the tool itself. Don't open or change it.
- `data_migration_helper.xlsx` — the mapping list. You fill this in.
- `requirements.txt` — a list of helper packages the tool needs. Used automatically during setup.
- `test_data` — practice files with made-up data, so you can try the tool safely before using it on real data.

# 2 Installation (One Time, On Each Computer)

Do this once on every computer that will run the tool. It takes about 5–10 minutes. You'll use the black Command Prompt window — don't worry, every step tells you exactly what to type.

## Step 2.1 — Check if Python is already installed

Click **Start**, type `cmd` and press Enter. A black window opens — this is the Command Prompt. Type:

```
python --version
```

- If it shows something like `Python 3.13.5` (3.8 or higher), Python is already installed — skip to Step 2.3.
- If it says *'python' is not recognized*, or the Microsoft Store opens, or the version is below 3.8, install Python using Step 2.2.

## Step 2.2 — Install Python (skip if Step 2.1 found it)

1. Go to `https://www.python.org/downloads/windows/`
2. Under the newest **Stable Release**, click **Windows installer (64-bit)**. Do not choose the "embeddable package" — it will not work for this tool.
3. Open the file you downloaded.
4. On the very first screen, **tick the box "Add python.exe to PATH"** at the bottom — this step matters. Then click **Install Now**.
5. When it finishes, click **Close**. If it offers **Disable path length limit**, click that too.
6. Close the Command Prompt window completely and open a new one. Type `python --version` again — it should now show a version number.

> **If the install asks for an administrator password**
>
> Ask your IT team to approve or run the installer. Don't skip the "Add to PATH" tick box — without it, none of the later steps will work.

> **If python --version still doesn't work, but py --version does**
>
> That's fine — Python installed without the PATH option. Just use `py` instead of `python` in Step 2.3 below (for example, `py -m venv .venv`). Everything else works exactly the same.

## Step 2.3 — Unzip the tool's folder

Right-click the zip file you received and choose **Extract All**. Pick a short, simple location — for example `C:\erp_data_migration` — it will be easier to type later.

## Step 2.4 — Open a Command Prompt inside the tool's folder

In File Explorer, open the tool's folder. Click once on the address bar at the top, type `cmd`, and press Enter. A black window opens, already inside that folder.

## Step 2.5 — Create the environment and install the packages

Type each of these three lines, pressing Enter after each one:

```
python -m venv .venv
.venv\Scripts\activate
python -m pip install -r requirements.txt
```

- After the second line, the prompt starts with `(.venv)` — that means the environment is switched on.
- The third line downloads the packages, so it needs an internet connection. It finishes with a line like `Successfully installed ... openpyxl ...`

> **Do this on every computer**
>
> A `.venv` folder copied from another computer will not work. Steps 2.3–2.5 must be repeated on each machine that will run the tool.

### If something goes wrong during installation

- **"No module named pip"** — type `python -m ensurepip --upgrade`, then run the install line again.
- **"'pip' is not recognized"** — type `python -m pip` instead of just `pip`. Make sure the line starts with `(.venv)` first; if not, run `.venv\Scripts\activate`.
- **Connection, proxy or SSL error** — the computer can't reach the internet to download packages. Check the connection, or ask IT if you're on a company network.
- **"A new release of pip is available"** — this is just a notice. You can ignore it.

# 3 Instructions — Running a Migration

## Step 3.1 — Fill in the mapping

Open `data_migration_helper.xlsx` and go to the **Mapping** sheet. Each row becomes one column of the new file, in the same order you list them:

- **New Column** — the column name the new system needs. Each name may only appear once.
- **Old Column** — the matching column name exactly as it appears in the old file. Capitalisation and extra spaces don't matter. Leave it blank to create an empty column in the new file. The same old column can feed more than one new column.
- **Notes** — for yourself only. The tool ignores this column.

> **Before your first real run**
>
> Delete the **EXAMPLE** rows. Don't rename the Mapping sheet or the column headers in row 1. Save and close the file before continuing.

## Step 3.2 — Run the tool

1. Close the old Excel file if it's open.
2. Open a Command Prompt in the tool's folder (Step 2.4) and switch the environment on — needed every time you open a new window:

```
.venv\Scripts\activate
```

Then run the tool in one of two ways:

### Way A — the tool asks you questions (easiest)

```
python main.py
```

It will ask for the old file, the sheet (only if there's more than one), the row with the column names, the mapping file, and the name of the new file. To give a file, simply drag it from File Explorer into the black window and press Enter. To accept a suggestion shown in [brackets], just press Enter. At the end it shows your answers and waits for Enter before starting.

### Way B — give all the details in one line

```
python main.py --old "C:\Data\old_data.xlsx" --output "C:\Data\new_data.xlsx"
```

- `--old` — the old file with the data.
- `--output` — the new file to create.
- Put **"quotes"** around every file path.
- To get a file's exact path: right-click it in File Explorer and choose **Copy as path** (hold Shift while right-clicking on older Windows), then paste it into the black window.

Add these extra options only when you need them:

- `--sheet "Customers"` — the old file has more than one sheet; give the name of the tab that has the data.
- `--header-row 3` — the column names aren't in row 1; give the correct row number.
- `--helper "C:\Data\other_mapping.xlsx"` — use a different mapping file instead of the one in the tool's folder.
- `--new-sheet "Import"` — name the sheet in the new file (default is Sheet1).
- `--overwrite` — replace the new file if it already exists. Without it, the tool stops instead of overwriting.

```
python main.py --old "C:\Data\old_data.xlsx" --sheet "Customers" --header-row 3
    --output "C:\Data\new_data.xlsx"
```

*Example: data on the “Customers” tab, column names on row 3.*

> **Want the full list of options explained?**
> Type `python main.py --help`

> **Never create the new file yourself**
> The tool creates the new Excel file. If a file with that name already exists, the tool stops rather than guessing what you want — add `--overwrite` if you really want to replace it.

## Step 3.3 — Check the result

When it finishes, the window shows a summary like this:

```
Old file     : C:\Data\old_data.xlsx (header row 1)
Columns      : 13 in new file, 12 filled from old file, 1 left empty
Rows copied  : 15
Verified     : 195 cells match the old file exactly
Saved to     : C:\Data\new_data.xlsx

New columns left empty (no Old Column in the helper): branch

Old columns NOT migrated (not in the helper): Sales Rep, Internal Notes
```

> **Always read the “NOT migrated” list**
>
> Those old columns did not make it into the new file because they weren't listed in the mapping. Double-check that is really what you want before relying on the result. Finally, open the new file and glance over a few rows yourself.

### Practice first with the test files

Before touching real data, try the tool on the made-up practice files in the test_data folder:

```
python main.py --old test_data\old_erp_data.xlsx --helper
    test_data\test_mapping_helper.xlsx --output test_data\new_erp_data.xlsx
```

Compare the result, `test_data\new_erp_data.xlsx`, against `test_data\expected_new_erp_data.xlsx` — they should match. The summary should report 15 rows, *branch* left empty, and *Sales Rep, Internal Notes* not migrated. To run the practice again, delete the output file first, or add `--overwrite`.

## 4 If You See an Error

When the tool stops with an error, it hasn't created anything yet. Fix the problem below and run it again.

- **‘python’ is not recognized, or the Microsoft Store opens** — the environment isn't switched on. Run `.venv\Scripts\activate` first. Still failing? Make sure you're in the tool's folder and that Step 2.5 was completed on this computer.
- **‘No module named openpyxl’** — same fix: run `.venv\Scripts\activate` first.
- **‘running scripts is disabled on this system’** — you’re in PowerShell, not Command Prompt. Type `cmd`, press Enter, and try again.
- **‘…already exists; use --overwrite’** — a file with that name already exists. Choose another name, or add `--overwrite`.
- **‘old column is not in row 1 … did you mean …’** — the Old Column name in the mapping is spelled differently from the old file. Fix the spelling, or add `--header-row` if the column names aren’t in row 1.
- **‘old column … appears more than once’** — two columns in the old file share a name. Rename one of them in a copy of the old file.
- **‘new column … is already used in row N’** — the same New Column name was typed twice in the mapping.
- **‘…has N sheets … choose one with --sheet’** — add `--sheet "SheetName"`.
- **‘cannot open … as an .xlsx file’** — the file is an old-style .xls file, or is damaged. Open it in Excel and use **Save As** → **Excel Workbook (.xlsx)**.
- **‘formulas whose result was never saved’** — open the old file in Excel, save it, close it, and run the tool again.
- **‘Permission denied … close the file if it is open in Excel’** — the new file is currently open in Excel. Close it and run again.

# 5 Safety — What the Tool Will Never Do

- It never changes the old file or the mapping file — both are only read.
- It checks every copied cell against the old file before saving. If anything doesn't match exactly, no new file is created.
- It never replaces an existing file unless you add `--overwrite`, or answer yes when asked.
- Its messages only ever show column names and cell positions (like `C14`) — never the actual data.
- Blank rows in the middle of the data are preserved in their original position; only blank rows at the very end are dropped.

---

*Questions about the mapping or a result you're not sure about? Ask before running the tool on real ERP data — it's always safer to double-check first.*